

Sustainable Life Management Model

Questions and Answers on Model Formulation and Solution Approach

February 4, 2026

Technical Documentation

Contents

1	Solution Methodology	2
2	Uncertainty Modeling	3
3	Key Concepts from Integer Programming	4
4	Summary	6

1 Solution Methodology

Question and Answer 1.1: Is the Model Solved via Dynamic Programming?

Question: Is the model solved in a dynamic programming way, i.e., forward then backward? What does the implementation codebase in `/src/` show?

Answer: No, the model is **not** solved using dynamic programming (forward-backward induction). Instead, it is formulated and solved as a **monolithic mixed-integer linear program (MILP)** using the Gurobi commercial solver.

Evidence from the codebase:

1. **Optimizer class** (`src/model/optimizer.py`): The model is built as a single Gurobi `Model` object that contains all decision variables for all time periods simultaneously:
 - All variables x_{it} , h_{it} , r_{ts} , z_{ts} for $t = 1, \dots, T$ are created at once
 - All constraints across all periods are added to one model
 - The objective function spans the entire horizon
2. **Solver class** (`src/model/solver.py`): The `Solve()` method simply calls `self.Model.optimize()`, which invokes Gurobi's branch-and-bound algorithm on the complete formulation.
3. **No stage decomposition:** There is no evidence of:
 - Value function computation
 - Backward induction over stages
 - State space enumeration
 - Bellman equation evaluation

Should it be solved via DP? The problem has characteristics that could theoretically support a DP approach (temporal structure, state variables g_t), but the current MILP formulation offers several advantages:

- **Tractability:** With $O(|\mathcal{I}| \cdot T \cdot |\mathcal{S}|)$ variables, the problem is small enough for direct MILP solution (achieves optimality within 60 seconds).
- **State space:** DP would require discretizing the continuous state space (G_t, σ_t) , introducing approximation errors.
- **Implementation simplicity:** MILP leverages mature commercial solvers without custom algorithm development.

2 Uncertainty Modeling

Question and Answer 2.1: How is Stochastic Uncertainty Considered?

Question: How is stochastic/uncertainty considered in the model beyond the definition/setting? How does it impact the solution process?

Answer: The model employs a **scenario-based stochastic programming approach** to handle uncertainty. The uncertainty is operationalized through the following mechanisms:

1. Scenario Set Definition:

- Discrete scenario set $\mathcal{S} = \{1, 2, 3, 4\}$
- Each scenario s specifies recovery hours $\beta_s \in \{1, 2, 3, 4\}$
- Probabilities $p_s = \{0.4, 0.3, 0.2, 0.1\}$

2. Impact on Objective Function:

The objective computes the **expected value** over scenarios:

$$\max \sum_{s \in \mathcal{S}} p_s \left[\sum_{t=1}^T \delta^t \sum_{i \in \mathcal{I}} U_{it}(h_{it}, g_t) \right]$$

From `src/model/objective.py`:

```
for Scen in range(config.SCENARIOS):
    Prob = config.RECOVERY_PROBS[Scen]
    Objective += Prob * (0.95 ** Day) * DayUtility
```

3. Impact on Constraints:

Scenario-indexed constraints ensure feasibility across all scenarios:

- **Time availability:** $\sum_{i \in \mathcal{I}} h_{it} + r_{ts} \leq H^{\max}$ for all t, s
- **Recovery triggering:** $r_{ts} \geq \beta_s \cdot z_{ts}$ for all t, s

This creates **scenario-dependent recovery requirements** while the primary allocation decisions h_{it} remain first-stage (here-and-now) variables that do not depend on which scenario realizes.

4. Solution Implications:

- Decisions must be robust to all recovery scenarios
- Worst-case scenario ($\beta_4 = 4$ hours) constrains maximum productive time
- Expected value objective balances performance across scenarios

Question and Answer 2.2: Where is Uncertainty Explicitly Modeled?

Question: Where in the model does it explicitly consider uncertainty and how is it modeled?

Answer: Uncertainty appears explicitly in three model components:

1. Stochastic Recovery Constraints (Lines 200-202 in Formulation.tex):

$$\sum_{i \in \mathcal{I}} m_i \cdot h_{it} \geq 0.8 \cdot M^{\max} - M \cdot (1 - z_{ts}) \quad \forall t \in \mathcal{T}, s \in \mathcal{S} \quad (1)$$

$$r_{ts} \geq \beta_s \cdot z_{ts} \quad \forall t \in \mathcal{T}, s \in \mathcal{S} \quad (2)$$

Interpretation:

- First constraint: If emotional load exceeds 80% of capacity, the binary indicator z_{ts} is forced to 1
- Second constraint: When $z_{ts} = 1$, recovery time r_{ts} must be at least β_s hours
- The big-M formulation models the logical implication: high emotional load $\Rightarrow$ recovery required

2. Time Availability with Scenario-Indexed Recovery:

$$\sum_{i \in \mathcal{I}} h_{it} + r_{ts} \leq H^{\max} \quad \forall t \in \mathcal{T}, s \in \mathcal{S}$$

This ensures that productive time plus scenario-dependent recovery does not exceed available hours.

3. Expected Value Objective:

$$\mathbb{E}_s \left[\sum_{t=1}^T \delta^t \sum_{i \in \mathcal{I}} U_{it}(h_{it}, g_t) \right] = \sum_{s \in \mathcal{S}} p_s \cdot \text{Utility under scenario } s$$

Model Classification:

This is a **two-stage stochastic program with recourse**:

- **First-stage decisions:** Task selections x_{it} and allocations h_{it} (made before uncertainty resolves)
- **Second-stage/recourse decisions:** Recovery hours r_{ts} (depend on scenario)
- **Relatively complete recourse:** Any first-stage decision can be made feasible by adjusting recovery

3 Key Concepts from Integer Programming

Key Concept 3.1: Mixed-Integer Linear Programming (MILP)

A MILP has the general form:

$$\begin{aligned} \max \quad & c^T x + d^T y \\ \text{s.t.} \quad & Ax + By \leq b \\ & x \in \mathbb{Z}^p, \quad y \in \mathbb{R}^q \end{aligned}$$

where x are integer variables and y are continuous variables.

Solution approaches:

- **Branch-and-bound:** Systematically explores the solution space using LP relaxations
- **Branch-and-cut:** Adds cutting planes to tighten LP relaxations
- **Dynamic programming:** Stage-wise optimization with value functions (not used here)

The SLM model uses MILP because:

1. Binary selection variables $x_{it} \in \{0, 1\}$ for task activation
2. Continuous allocation variables $h_{it} \in \mathbb{R}_+$ for hours
3. Linear objective and constraints enable efficient solution

Key Concept 3.2: Stochastic Programming

Stochastic programming optimizes decisions under uncertainty:

Two-stage formulation:

$$\begin{aligned} \min_x \quad & c^T x + \mathbb{E}_\xi[Q(x, \xi)] \\ \text{s.t.} \quad & Ax = b, \quad x \geq 0 \end{aligned}$$

where $Q(x, \xi)$ is the optimal value of the second-stage problem given first-stage decision x and scenario ξ .

Scenario-based approximation:

Replace expectation with weighted sum over discrete scenarios:

$$\mathbb{E}_\xi[Q(x, \xi)] \approx \sum_{s \in \mathcal{S}} p_s \cdot Q(x, s)$$

This converts the stochastic program to a deterministic equivalent problem (DEP), which is exactly what the SLM formulation implements.

Key Concept 3.3: Dynamic Programming vs. MILP

Dynamic Programming:

- Decomposes problem by stages/time periods
- Computes value functions via backward induction
- Optimal policy obtained by forward simulation
- Suffers from “curse of dimensionality” with continuous states

Bellman equation:

$$V_t(s) = \max_{a \in A(s)} \left\{ r(s, a) + \gamma \sum_{s'} P(s'|s, a) V_{t+1}(s') \right\}$$

MILP approach:

- Formulates entire horizon as single optimization

- Exploits problem structure via branch-and-bound
- Handles continuous variables naturally
- Leverages commercial solver technology

When to use each:

- **DP:** Large discrete state spaces, Markov structure, need for policy
- **MILP:** Moderate problem size, mixed integer-continuous variables, commercial solver access

The SLM problem falls into the MILP-favorable category due to continuous state variables and tractable problem size.

4 Summary

Key Concept 4.1: Key Takeaways

1. **Solution method:** The SLM model is solved as a monolithic MILP using Gurobi's branch-and-bound algorithm, **not** via dynamic programming.
2. **Uncertainty handling:** Scenario-based stochastic programming with discrete scenarios for recovery requirements. The objective maximizes expected utility, and constraints ensure feasibility across all scenarios.
3. **Explicit uncertainty locations:**
 - Scenario-indexed recovery variables r_{ts}
 - Binary high-load indicators z_{ts}
 - Time availability constraints
 - Expected value objective
4. **Two-stage structure:** First-stage (here-and-now) decisions for task allocation; second-stage (recourse) decisions for recovery adjustment.
5. **Tractability:** Problem size $O(|\mathcal{I}| \cdot T \cdot |\mathcal{S}|)$ is well-suited for direct MILP solution without decomposition.